Part 1: Individual Cloud API (Required)

- **API Name :** Overall Progress API
- **Purpose :** This API displays study activity, number of summaries/quizzes/flashcards generated, score performance over time, and recent quizzes or flashcards.
- **Method (GET, POST, PUT, DELETE) :** GET
- **Cloud Service (e.g., Azure, AWS, etc.) :** Google Firestore
- Output :


```json
{
  "stats": {
    "totalQuizzes": 12,
    "averageScore": 86,
    "totalSummaries": 9,
    "totalFlashcards": 134
  },
  "scoreTrend": [
    { "date": "2026-04-10", "score": 75 },
    { "date": "2026-04-12", "score": 90 }
  ],
  "activity": [
    { "date": "2026-04-10", "count": 2 },
    { "date": "2026-04-11", "count": 1 }
  ],
  "recent": [
    {
      "type": "quiz",
      "title": "Photosynthesis",
      "score": 92,
      "date": "2026-04-15"
    },
    {
      "type": "flashcards",
      "title": "Photosynthesis",
      "count": 8,
      "date": "2026-04-14"
    }
  ]
}
```


- Related Module : Progress Tracking Module
- Related Database Table(s) : Progress

Part 2: Team Cloud API

API 1: Text Summarization API

- **API Name :** Text Summarization API
- **Purpose :** This API converts user provided text or uploaded content into a concise summary to help students quickly understand key concepts
- **Method (GET, POST, PUT, DELETE) :** POST
- **Cloud Service (e.g., Azure, AWS, etc.) :** Deployed on Vercel
- **Input**

Format: multipart/form-data

Key	Type	Description
text	String	Text to summarize
file	File (optional)	Uploaded document
mode	String	Operation type (e.g., “summary”)

- **Output**

```
{
"result": "Artificial Intelligence helps automate tasks and improve efficiency..."
}
```

- Related Module : Summarization AI Agent
- Related Database Table(s) : Content & Summary

API 2: Video Summarization API

- **API Name :** Video Summarization API
- **Purpose :** This API extracts and summarizes key information from video content (such as lectures or YouTube videos) to help students review material efficiently.
- **Method (GET, POST, PUT, DELETE) :** POST
- **Cloud Service (e.g., Azure, AWS, etc.) :** Deployed on Vercel
- **Input**

  Format: JSON


```
{
    "url": "https://www.youtube.com/watch?v=example"
}
```


- **Output**


```
{
"summary": "This video explains key concepts of Artificial Intelligence including automation and efficiency..."
}
```


- **Related Module :** Video/Lecture Summarization Agent

- **Related Database Table(s) :** Content, Summary & Video

Part 3: Request & Response

API 1: Text Summarization API

Endpoint:

https://summarize-lemon.vercel.app/api/summarize

Method:

POST

Request Format:

This API uses form - data (NOT JSON)

Body

Key	**Value**
text	Any paragraph you want to summarize
mode	summary

Example Input

text :

Artificial Intelligence helps automate tasks and improve efficiency.

mode :

summary

Expected Output :

```
{
  "result": "Artificial Intelligence helps automate tasks and improve efficiency by improving productivity and reducing manual work."
}
```

Error Case 1: Missing Text

```json
{
  "error": "Text input is required"
}
```

Eros Case 2: Wrong Format (JSON instead of form-data)

```json
{
  "error": "Invalid request format"
}
```

API 2: Video Summarization API

Endpoint:

https://summarize-lemon.vercel.app/api/video-summarize

Method:

POST

Request Format:

JOSN

Request Body

```
{
  "url": "https://www.youtube.com/watch?v=example"
}
```

Example Input

url: https://www.youtube.com/watch?v=example

Expected Output :

```
{

  "summary": "This video explains how Artificial Intelligence works and how it improves efficiency
through automation and data processing."

}
```

Error Case 1: Missing URL

```json
{
  "error": "Video URL is required"
}
```

Eros Case 2: Invalid or Unsupported Video

```json
{
  "error": "Unable to extract transcript from video"
}
```

Part 4: Peer API Testing (In-Class Activity)

Each team will test another team's APIs AS WE LINED UP LAST WEEK

During the session:

- Review assigned team's APIs
- Check clarity of input/output
- Validate logic and correctness
- Test edge/error cases

Part 5: Testing Results

Team Tested: Game Start API Team **APIs Tested:** Games List API — `GET https://game-start-api-v1.onrender.com/api/games`

Games List API

What is good

- Endpoint is live, returns valid JSON, no auth needed — easy to test.
- Response includes solid game data: name, rating, platforms, genres, screenshots, etc.
- Includes pagination (`count`, `next`, `previous`) and uses proper GET method.

Issues found

- **Exposed API key.** The `next` URL leaks their RAWG API key in plaintext — needs to be fixed ASAP.
- **It's just a RAWG proxy.** Response is passed through unchanged with no real value-add layer.
- **No documentation.** No input fields, error cases, or sample requests provided.
- **Response is bloated.** Returns full system requirements and HTML for every game in the list.
- **No error cases testable.** Only the root endpoint was shared.

Suggestions for improvement

1. Hide the RAWG API key server-side and rewrite pagination URLs to their own domain.
2. Add query params for filtering/searching (`?search=`, `?genre=`, `?platform=`).
3. Add a detail endpoint (`GET /api/games/:id`) so there's more than one route.
4. Trim the list response — move heavy data to the detail endpoint.
5. Document the API properly with sample requests, responses, and error cases.